Key Management, Distribution, and Public Key Infrastructures

Jean-Michel Dricot – ULB Cybersecurity Research Center

In This Lecture

- Key management and deployment
- Internet model for key exchange
- Public Key Infrastructure (PKI)
- Certificate/Key validation techniques
- Certificate/Key Revocation

Cryptographic Key Management

- The secure use of cryptographic key algorithms depends on the protection of the cryptographic keys

Cryptographic key management

Cryptographic key management is the process of administering or managing cryptographic keys for a cryptographic system

- It involves the generation, creation, protection, storage, exchange, replacement, and use of keys and enables selective restriction for certain keys
- A key management system will also include key servers, user procedures, and protocols

Key Distribution Technique

Key Distribution

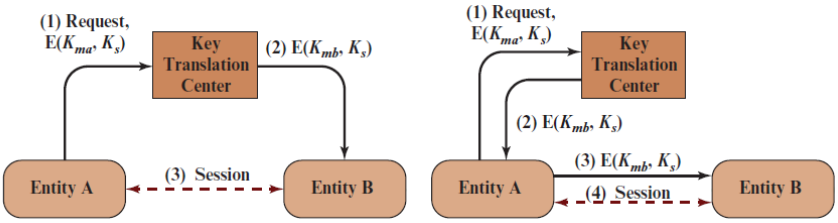
Means of delivering a key to two parties who wish to exchange data without allowing others to see the key

- For symmetric encryption to work, the two parties to an exchange must share the same key, and that key must be protected from access by others
- Frequent key changes are desirable to limit the amount of data compromised if an attacker learns the key

Symmetric Key Distribution

- Given parties A and B, key distribution can be achieved in a number of ways:
  - A can select a key and physically deliver it to B
  - A third party can select the key and physically deliver it to A and B
- If A and B have previously and recently used a key, one party can transmit the new key to the other, encrypted using the old key
- If A and B each has an encrypted connection to a third party C, C can deliver a key on the encrypted links to A and B

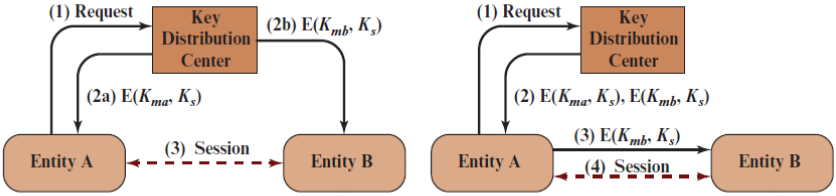
Key Distribution Between Two Entities



(a) Key Translation

(b) Key Translation with Key Forwarding

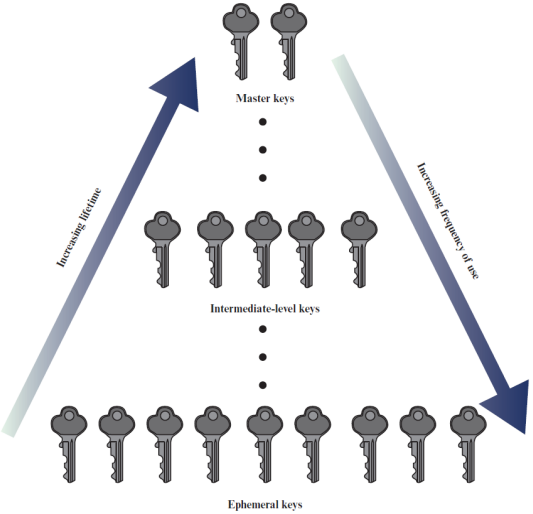
Key Distribution Between Two Entities



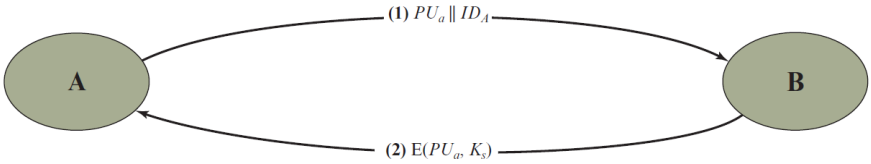
(c) Key Distribution

(d) Key Distribution with Key Forwarding

Symmetric Key Hierarchy

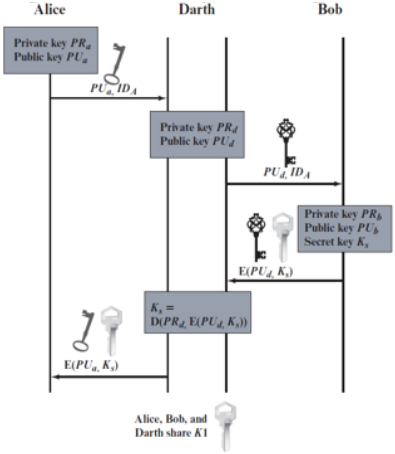


Simple Use of Public-Key Encryption to Establish a Session Key

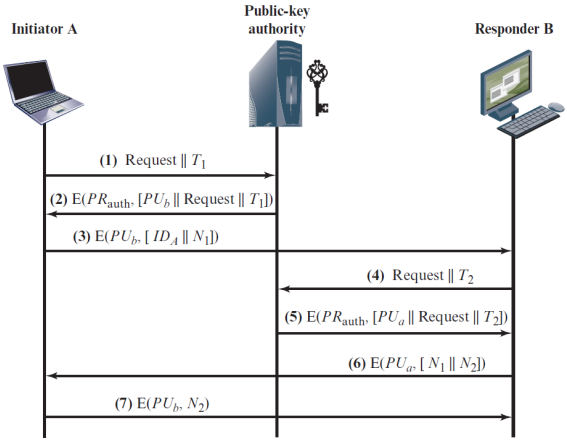


Simple Use of Public-Key Encryption to Establish a Session Key

. . . but just another Man-in-the-Middle Attack



Public-Key Distribution Scenario



Why we need PKIs

- How to establish "trust" in a public key:
  - Alice and Bob first exchange their keys via some method; then they validate if they have received the correct keys via a secure (!) channel
  - Secure channel: personal meeting, video chat, phone call, ...
  - Extra cautious persons will also identify each other using a official ID document
  - High effort!
- How to scale this approach to large networks like ... the Internet?

Why we need PKIs

- How to scale this approach to large networks like ... the Internet?
- Use a PKI instead; simplified, generic idea:
  - "Notary" verifies the identity of the entity who owns a public/private key pair (and verifies additional info like the correctness of this entities mail address)
  - Notary creates data structure that binds the public key to the identity of the entity (+ additional info) and certificate
  - Notary signs the certificate with her private key to make it verifiable
  - Who ever wants to use the cert and trusts the notaries assertions and can verify her signatures can trust the public key contained in the certificate

Certificates: The essence of PKI

Definition

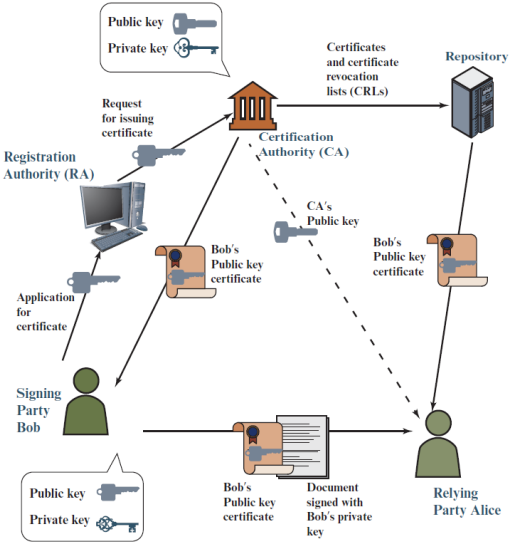
A certificate is a cryptographic binding between an identifier and a public key that is to be associated to that identifier.

Semantics of the binding

- The identifier often refers to a person, business, etc. While much less common, the identifier may also indicate some attribute with which the key is associated (e.g., access right).
- Always necessary: Verification that identifier and corresponding key belong together.
- If the identifier is a name: verify that the entity behind the name is the entity it claims to be

Chains can be established : E1 may certify E2, who certifies X: E1 -> E2 -> X.

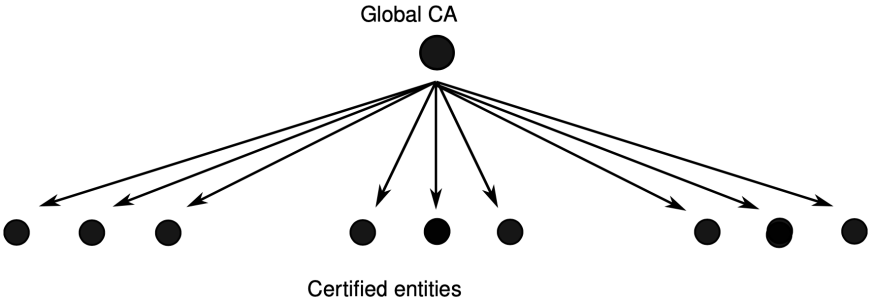
PKI Scenario



Common forms of PKI

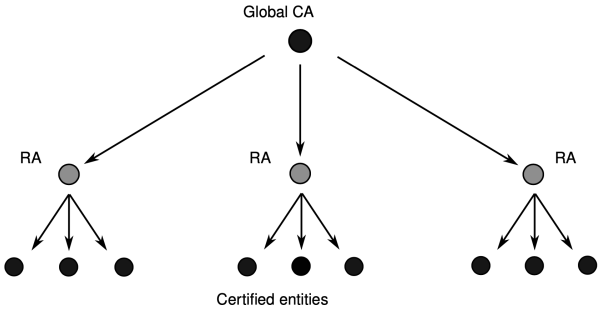
- We can now classify PKIs by looking at:
  - Who are the issuers?
  - Which issuers must be trusted = which TTPs exist?
  - How do issuers verify that X and KX-pub belong together, or that X is really X?
- Some terminology
  - Depending on the PKI, different words for "issuer" exist
  - Often in hierarchical PKIs: "Certification Authority" (CA)
  - In non-hierarchical PKIs sometimes: "Endorser"
  - These words often hint at the role (power) of the issuers

Hierarchical PKIs – naive form



- This is an infeasible form.
  - Who decides which global authority is trustworthy for the job?
  - What are the agreed verification steps?
  - This, and the high load on the CA, may make it easier to trick the CA into misissuing a certificate to, e.g., wrong entity
  - Hard to imagine any government would rely on an authority outside its legal reach.

Hierarchical PKIs – Intermediate authorities

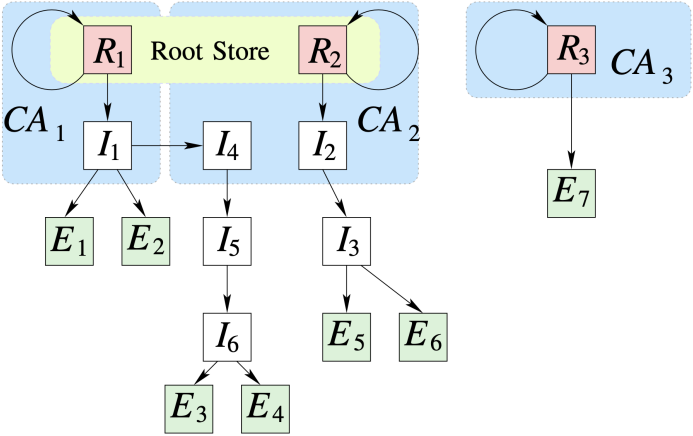


- Scales betters but remains centralized
  - RAs do not issue certificates : they are mere proxies
  - Can perform the verification steps
  - Can react differently, according to the local regulation

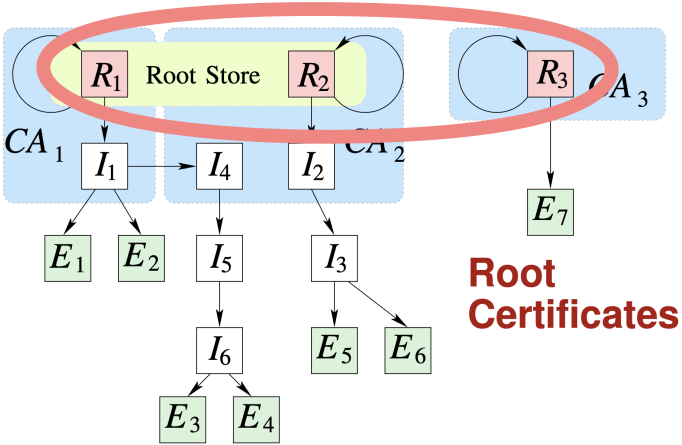
Hierarchical PKIs – Current solution

- Intermediate RA for scaling
- Many global CAs
  - One global CA is infeasible, even with RAs
  - Use many CAs, in different legislations,
  - Should we accept them all equally? **No**.
- Defining CAs as trusted
  - A CA must be trusted by participants in order to be useful
  - How should participants decide which CAs to trust?
  - "Solution": operating systems and software like browsers come preconfigured with a set of trusted CAs

Hierarchical PKIs – Current solution



Hierarchical PKIs – Root stores

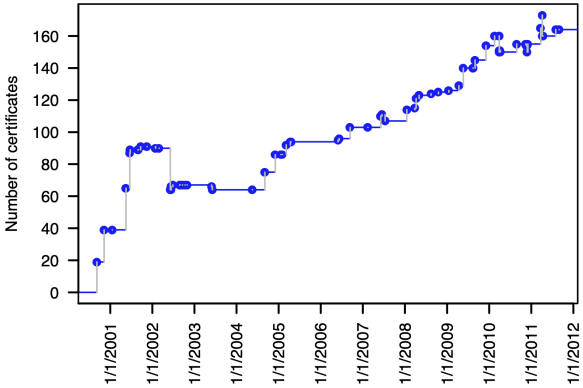


Hierarchical PKIs – Root stores

- Root stores: certificates of trusted CAs
  - 'Trusted' = trusted to issue certificates to the correct entities
  - Every application that uses X.509 has to have a root store
  - Operating Systems have root stores: Windows, Apple, Linux
  - Browsers use root stores: Mozilla ships their own, IE uses Windows' root store, etc.
- Root store processes
  - Every root store vendor has their own process to determine if a CA is added or not
  - A CA's Certification Policy Statements (CPS) are assessed
  - Mozilla: open discussion forum (but very few participants)
  - Commercial vendors (Microsoft, Apple): little to no openness

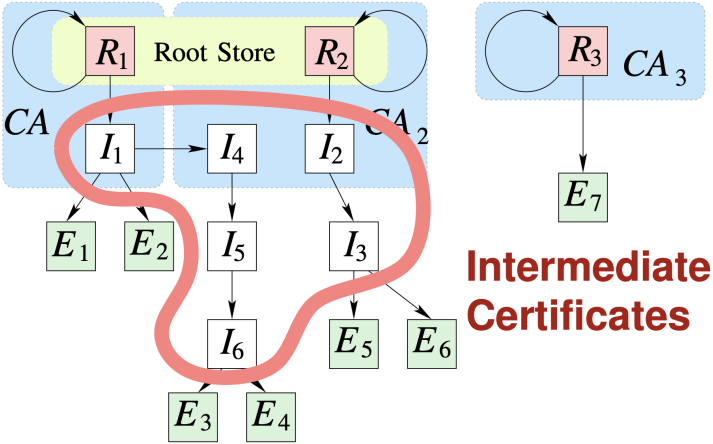
Hierarchical PKIs – Root stores

Development of Mozilla Root Store



(Dec. 2018: 164 root certificates in store)

Hierarchical PKIs – Intermediate Certificates



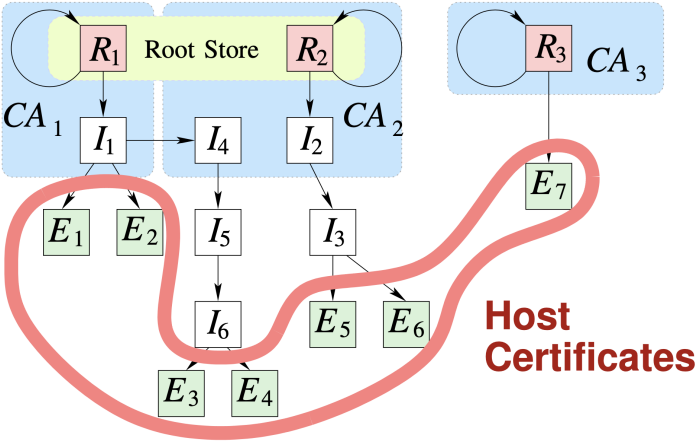
Hierarchical PKIs – Intermediate Certificates

- There are two primary reasons to use intermediate certificates:
  - To delegate signing authority to another organisation: sub-CA
- Protect your main root certificate:
  - Intermediate cert is operated by the same organisation
  - Allows to store root cert in the root store, but private key may remain offline in some secure location
  - Online day-to-day operations can be done using the private key of the intermediate cert
  - Also makes it very easy to replace the intermediate cert in case of compromise, or crypto breakthroughs (e.g. hash algorithms) etc.

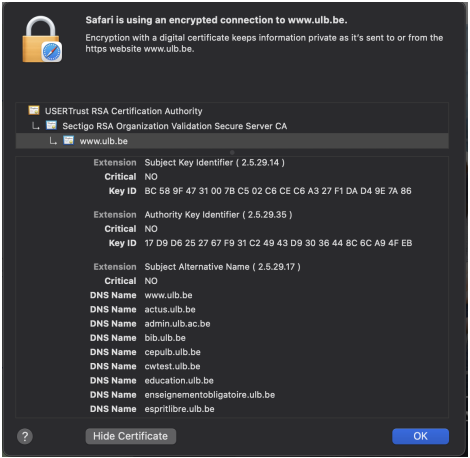
Hierarchical PKIs – Hazards of Intermediate Certificates

- Some companies/organisations have SSL proxies
  - They monitor their employees' traffic
  - May make sense in order to avert things like industrial espionage
  - However, some CAs have issued intermediate certs to be used as sub-CAs in proxies or added to client root stores
  - This allows transparent rewriting of certificate chains– a classic Man-in-the-middle attack
  - Worst: the holder of the sub-CA is suddenly as powerful as all CAs in the root store
  - Since outing of first such CA, Mozilla requires practice to be disclosed, and stopped

Hierarchical PKIs – Host Certificates



Hierarchical PKIs – Host Certificates



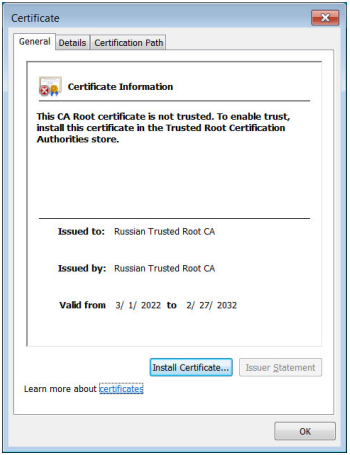
Current PKIs Issues

Remember...

- Your browser or your OS chooses the 'trusted CAs'. **Not you**.
- All CAs have equal signing authority (there are efforts to change this)
- **Any CA may issue a certificate for any domain.**
- DNS path restrictions are a possibility; must be set by the CA in their signing cert
- A globally operating CA cannot feasibly set such restrictions in their root cert

Current PKIs Issues

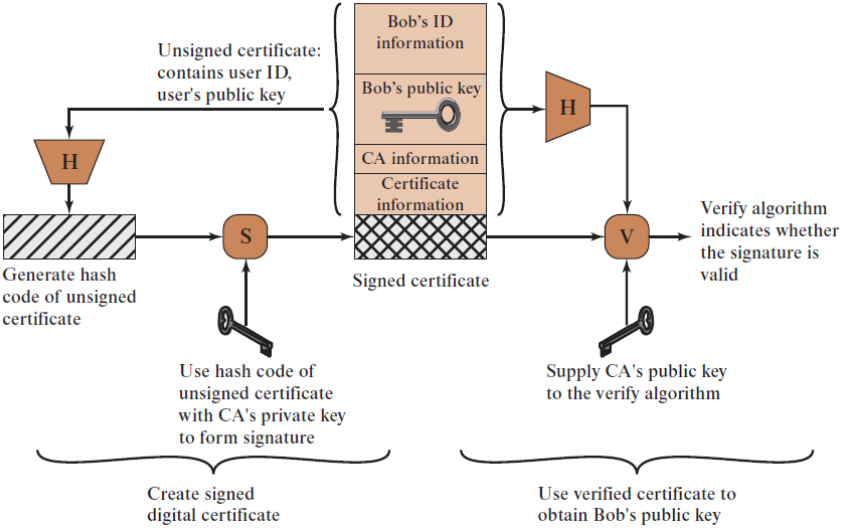
March 2022 : *Russia creates certificate authority in response to sanctions*



X.509 Certificates

- Part of the X.500 series of recommendations that define a directory service
  - The directory is, in effect, a server or distributed set of servers that maintains a database of information about users
- X.509 defines a framework for the provision of authentication services by the X.500 directory to its users
  - Was initially issued in 1988 with the latest revision in 2016
  - Based on the use of public-key cryptography and digital signatures
  - Does not dictate the use of a specific algorithm but recommends RSA
  - Does not dictate a specific hash algorithm
- Each certificate contains the public key of a user and is signed with the private key of a trusted certification authority
- X.509 defines alternative authentication protocols based on the use of public-key certificates

X.509 Certificates

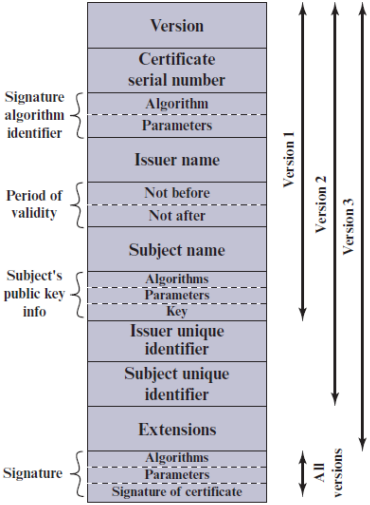


X.509 Certificates

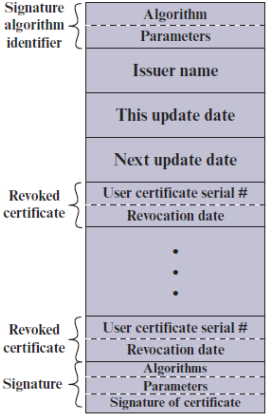
Created by a trusted Certification Authority (CA) and have the following elements:

- Version
- Serial number
- Signature algorithm identifier
- Issuer name
- Period of validity
- Subject name
- Subject's public-key information
- Issuer unique identifier
- Subject unique identifier
- Signature

X.509 Certificates



(a) X.509 certificate



(b) Certificate revocation list

Certificate Issuance

- How is a certificate issued in practice?
  - Domain Validation (DV): proves ownership of the domain
  - Publish a CA-specified nonce in a DNS TXT record
  - Publish a CA-specified nonce on the web server
  - CA sends mail to mail address specified in domain's whois info; respond to this mail
  - CA sends mail to "protected" mail address (admin@...); respond to this mail
  - Note: Let's Encrypt issues DV certificates
- Extended Validation (EV):
  - Additionally requires (strong) legal documentation of the claimed identity
- Organisational Validation (rare):
  - Between DV and EV; less documentation

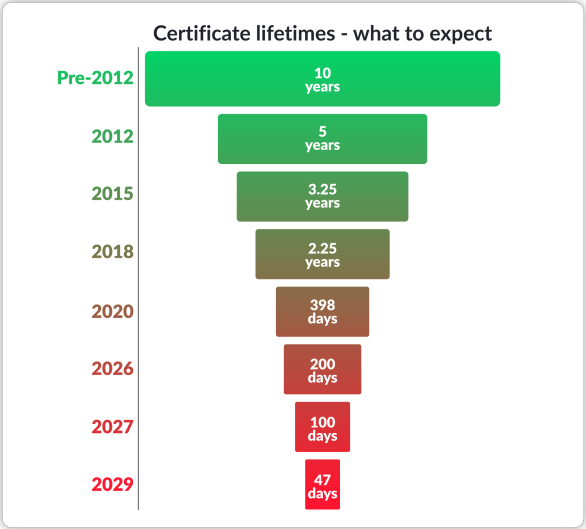
Certificate Issuance – SSL toolbox

- Standard approach of domain validation:
  - Generate RSA key
  - Generate Certificate Signing Request (CSR)
  - Log into the CA's awesome webinterface
  - Fill out the ordering form, upload CSR
  - Verify domain with one of many vendor-specific verification methods (Mail to ca-admin@domain, http- Textfile, TXT DNS record, . . .)
  - Pay some sum of money
  - Receive and install the certificate

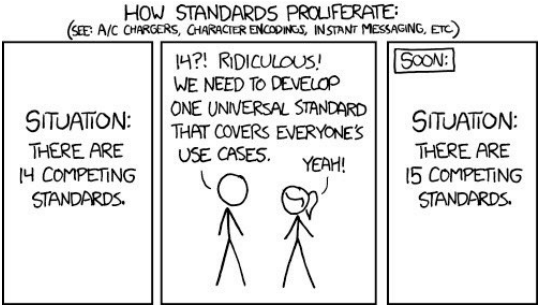
Certificate Issuance – Economics and security

- PKI is a good area to study dynamics and interplay of economics and security
- Incentive to lower prices -> less checks, makes certification cheaper
- Actually not true! Results of a study :
  - Empirical (quantitative) part: **the more expensive CAs have more customers**
  - Quantitative part: in interviews, customers say they prefer a CA that is **'too big to fail'** and will never be **removed from root stores**
  - Indeed, large CAs are difficult to remove from root stores as the Web browser would suddenly show errors for many sites!
- **This shows customers behave rationally correct**, but different from what designers of security system would have expected

Certificate lifetime constraints



Could we fix all of this ?



Certificate Issuance Automatization

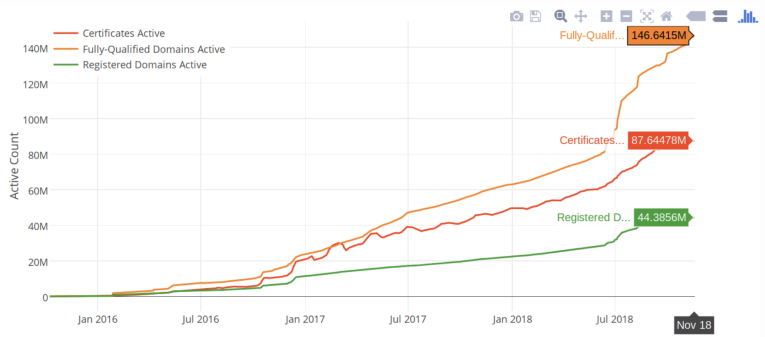
ACME: Automated Certificate Management Environment

- JSON protocol for unsupervised certificate issuance
  - Definition as IETF draft on https://ietf- wg- acme.github.io/acme/
  - Used by Let's Encrypt https://letsencrypt.org
  - **Domain validated certificates only**
- ACME Client creates an "account key" (asymmetric cryptography keypair)
  - Can now send signed messages with JSON Web Signature (JWS) (RFC7515)
  - Required to register account: [ToS Agreement, contact info] signed with client privkey
- How does the certificate issuance process work?
  - Step 1: authorize account key for domain
  - Step 2: request server certificate for this domain

Let's Encrypt

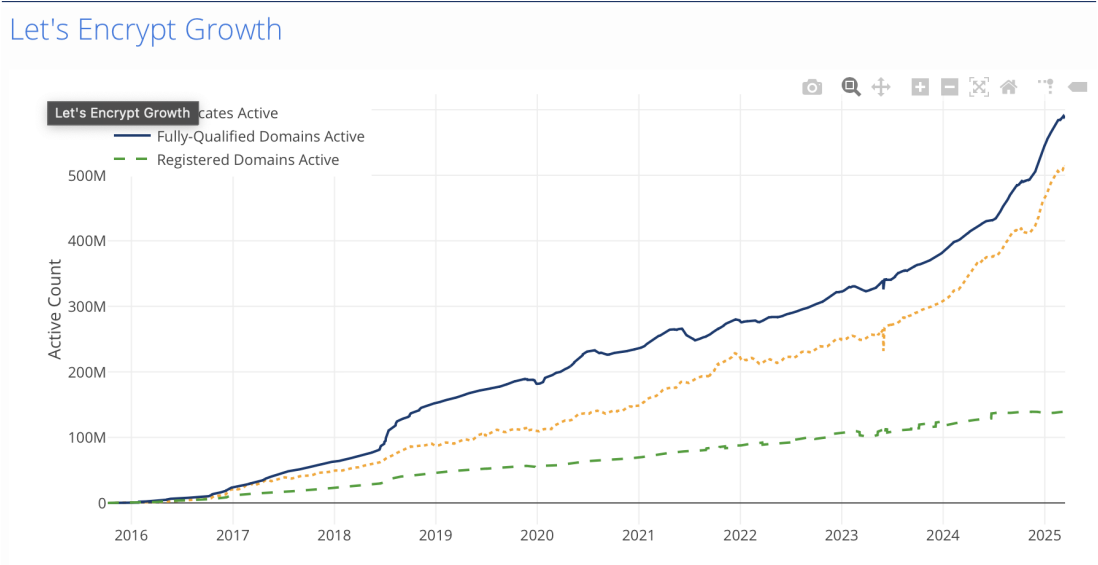
- Free, automated, and open certificate authority (CA), run for the public's benefit
- By Internet Security Research Group (ISRG)
- ISRG: California public benefit corporation, backed by Akamai, ACLU, Google, Mozilla, Cisco, EFF, CoreOS, OVH, Facebook and more.
- Mission: Reduce financial, technological, and educational barriers to secure communication over the Internet

Let's Encrypt



- Rate limits (evolving):
  - 50 Certificates per Registered Domain per week, renewals always okay
  - 100 Names per Certificate (SAN)
  - Duplicate Certificate limit of 5 certificates per week
  - Failed Validation limit of 5 failures per account, per hostname, per hour

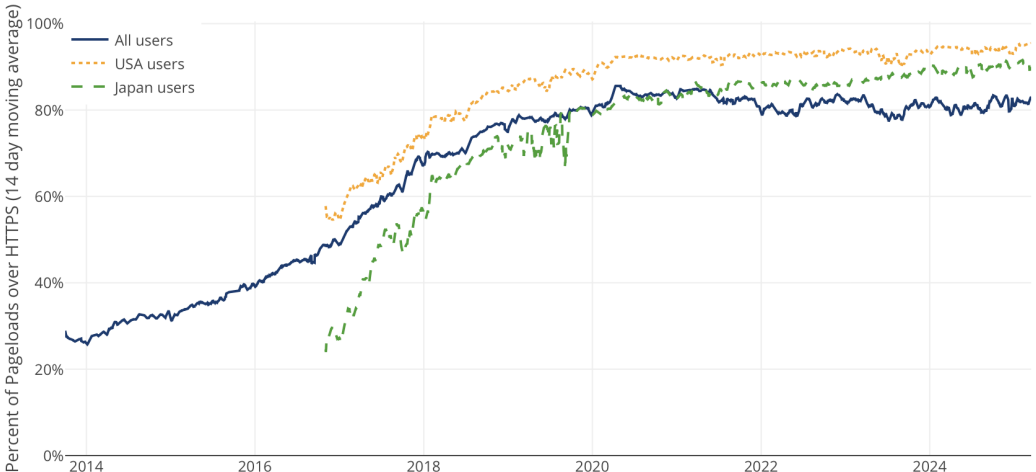
Let's Encrypt adoption



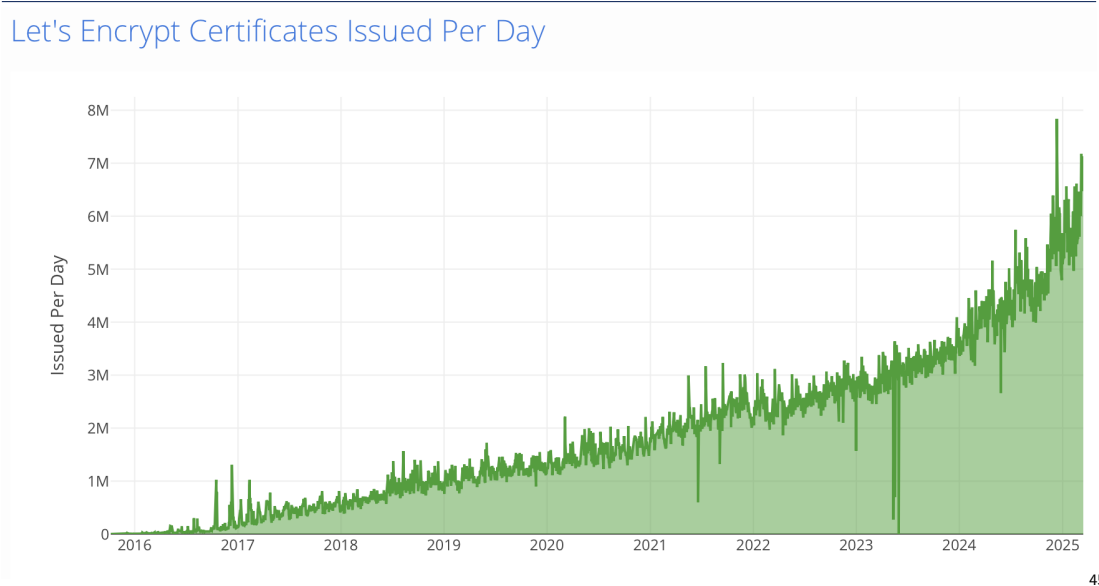
Let's Encrypt adoption

Percentage of Web Pages Loaded by Firefox Using HTTPS

(14-day moving average, source: Firefox Telemetry)

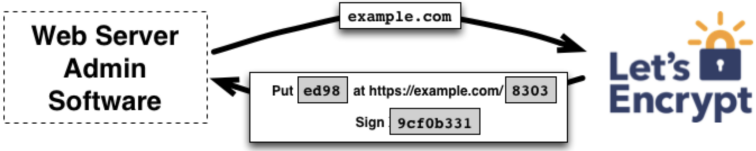


Let's Encrypt adoption



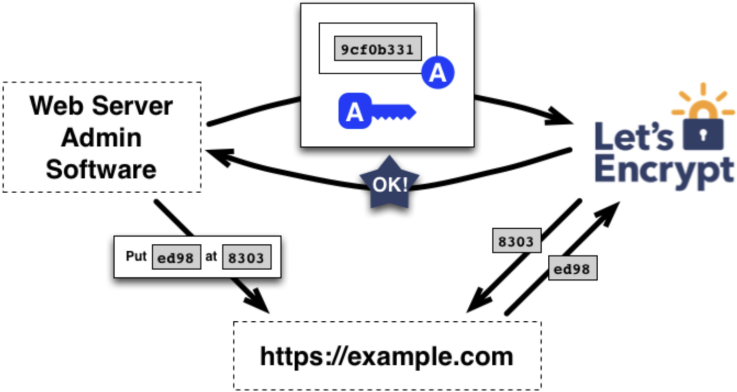
Let's Encrypt

Client requests authorization and receives challenge (*HTTP-01 challenge*)



Let's Encrypt

Client signs and publishes challenge. The CA can now check the published file content:



Challenge and file content ok : Account key pair is then "authorized"

Let's Encrypt

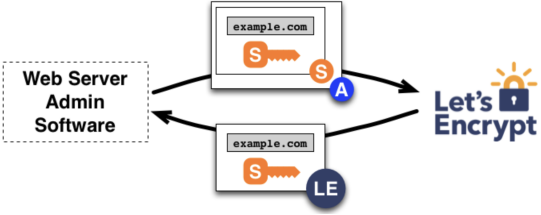
Alternate method based on DNS (*DNS-01 Challenge Verification*)

- CA requests entry of DNS record
  - Client constructs key authorization from token offered by server challenge and the client's account key
  - Generate SHA-256 digest in base64
  - Store as DNS TXT record under `_acme-challenge` for domain to verify
    `_acme-challenge.example.org.  300 IN TXT "gfj9Xq...Rg85nM"`
- Challenge ok -> Account key pair is then "authorized"

Let's Encrypt

Lifecycle : renewal

- Authorized account key pair can request, renew and revoke certificates for the domain by signing messages
  - Create Certificate Signing Request (CSR) (RFC2986) is created for desired domains
  - CSR is signed by private key of the public key that's also in the CSR
- Account key also signs whole certificate request with its private key
  - The CA validates both signatures, checks ratelimit, and signs the request:

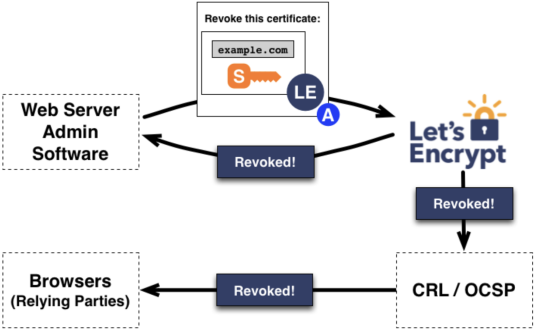


Let's Encrypt

Lifecycle : revocation

When a certificate shall be revoked, sign the revocation request with the authorized keypair!

The revocation is processed by the CA and distributed to clients.



Certificate Revocation

Revocation is crucial—yet often neglected in discussions

- Problem: Certificates typically have a specified lifetime; X.509: Not Before / Not After fields
- We need confirmation that a certificate is valid at the moment of interest, not some time in the past
- There are several cases when an already issued certificate must be withdrawn. Examples:
  - Corresponding private key compromised
  - Certificate owner does not operate service any longer
  - Key ownership has changed
- In these cases, there are two options: CRLs and OCSP
- No certificate can be considered valid without a revocation check

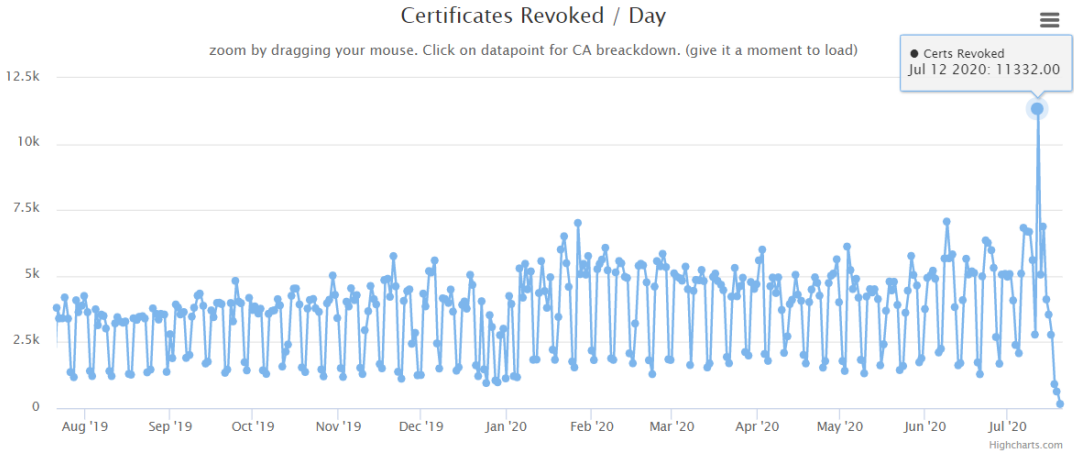
Certificate Revocation Lists (CRLs)

A CRL is a list of certificates that are considered revoked

- They are (should be) issued, updated and maintained by every CA
  - Certificates are identified by serial number
  - A reason for revocation can be given
  - Every CRL must be timestamped and signed
- There are further entries, like time of next update
- Technically, a browser (client) should download CRL (and update it after the given time), and lookup a host certificate every time it connects to a server

Certificate Revocation Lists (CRLs)

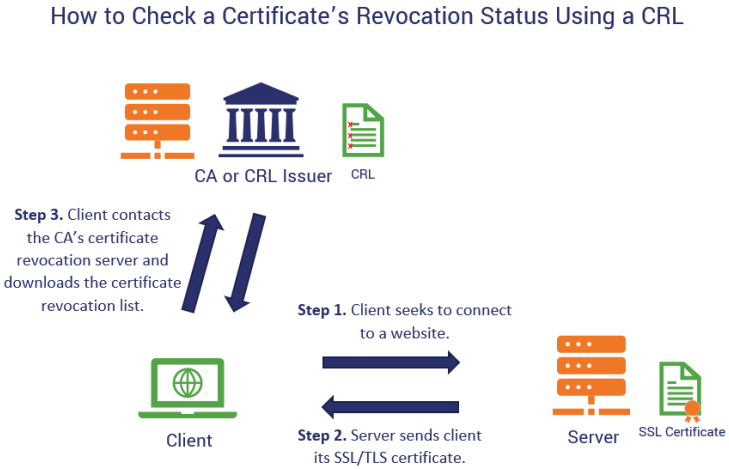
Certificates Revoked per Day



Certificate Revocation Lists (CRLs)

```
X509 Certificate Revocation List:
Version: 2
Signature Algorithm:
    Algorithm ObjectId: 1.2.840.113549.1.1.11 sha256RSA
    Algorithm Parameters:
    05 00
Issuer:
    CN=COMODO RSA Extended Validation Secure Server CA
    O=COMODO CA Limited
    L=Salford
    S=Greater Manchester
    C=GB
  Name Hash(sha1): f4317077aef7881259dd9e5d23f2fe267766d046
  Name Hash(md5): e9a98686b149c459ed86ebd8769dbd0e

 ThisUpdate: 7/21/2020 3:01 PM
 NextUpdate: 7/28/2020 3:01 PM
CRL Entries: 2897
  Serial Number: deb98d79ccac41cd521f10cd05342998
   Revocation Date: 6/11/2018 4:19 AM

  Serial Number: 6c924d99c346d004f912b9aa5b0a61dc
   Revocation Date: 6/14/2018 3:40 AM

  Serial Number: 7d8eb0779e43059486bc6ae9260a3fcc
   Revocation Date: 6/14/2018 4:05 AM

  Serial Number: aebc9726ddaa1b6a279ef7cdca800f64
   Revocation Date: 6/14/2018 4:05 AM

  Serial Number: cf345c3c12f2b35a40ce2f6c8cc85164
   Revocation Date: 6/19/2018 1:26 PM
```

Certificate Revocation Lists (CRLs)



Problems with CRLs

CRLs have a number of problems

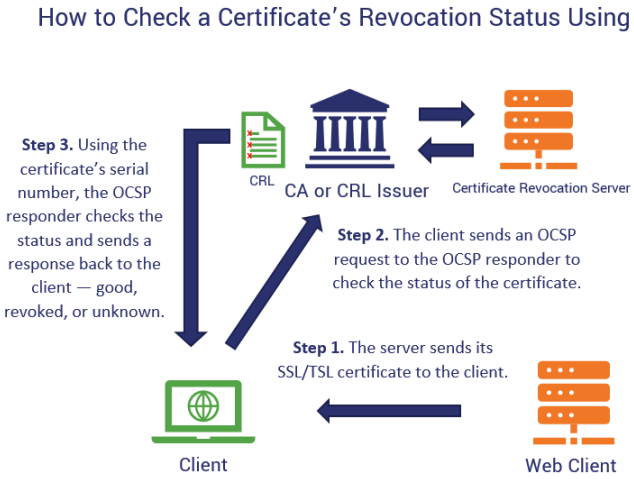
- Intermediate certs should be checked, too – induces load and network activity
- There is a time interval between two updates (window for attack)
- CRLs can grow large
  - Response to this: Delta CRLs that contain only latest updates
  - Requires server side support—very rarely used
- Downloads of CRLs can be blocked by a Man-in-the-middle
- For these reasons, browsers have never activated CRLs by default

Online Certificate Status Protocol (OCSP)

OCSP allows live revocation checks over the network

- Query-response model
- Query = lookup of a certificate in a server-side CRL-like data structure
  - Query by several hash values and cert's serial number
  - Replay protection with nonces
  - Query may be signed
  - Does not require encryption
- Response:
  - Contains cert status: good, revoked, unknown
  - Must be signed

Online Certificate Status Protocol (OCSP)



New Approaches to Revocation

In-browser / In-App revocation lists

- Browsers preload a list of revoked certificates for the most common and important domains
- Updates are distributed via the browser's update mechanism
- This counters the devastating attacks where traffic to the CA is dropped—but the scalability is not good

Short-lived certificates

- Give certificates a very short validity period (1 hour–1 day)
- Replace certificates fast, do not attempt any other revocation
- Works well and gives very clearly defined window of attack
- Problem: certification becomes a frequent and 'live' operation

(Static) certificate pinning

Certificate Pinning (TOFU)

Aim: reassurance of a certificate's authenticity

- As a defence against rogue CAs issuing malicious certs
- **Idea: client stores information about a host/Web site on first contact**
- Most commonly: store the public key of a site
- Use this information to re-identify a site later
- E.g. if public key is suddenly different on next connect: warn user

Pinning assumes a secure first connection

- Thus also known as 'trust-on-first-use' (TOFU)
- Inherent bootstrapping problem

Full static pinning (in-app certificate)

- suppy chain management security (and *secure* App stores model)